

Dynamic Programming: DP on Stocks

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Dynamic Programming: DP on Stocks	1
1.1	The Holding-State Template	1
1.2	Best Time to Buy and Sell Stock I	2
1.3	Best Time to Buy and Sell Stock II	5
1.4	Best Time to Buy and Sell Stock III	7
1.5	Best Time to Buy and Sell Stock IV	10
1.6	Best Time to Buy and Sell Stock with Cooldown	13
1.7	Best Time to Buy and Sell Stock with Transaction Fee	17

Chapter 1

Dynamic Programming: DP on Stocks

Every stock problem in this chapter asks the same question on every day: given whether I am currently holding a share or not, should I act today, or wait? Once that one binary state is in the recurrence, every variant in this chapter – a transaction limit, a cooldown, a fee – is a one-line adjustment to the same two equations.

1.1 The Holding-State Template

Every problem in this chapter shares one underlying state machine: on any given day, you are either **holding** a share (bought earlier, not yet sold) or **not holding** one. The DP state is $(day, holding)$, and the recurrence at every day asks exactly one question: act (buy, if not holding; sell, if holding), or wait and carry the same holding-state into tomorrow?

The Base Recurrence Every Variant in This Chapter Modifies

$$\begin{aligned} f(day, 0) &= \max(-price[day] + f(day+1, 1), f(day+1, 0)) \quad (\text{not holding: buy or wait}), \\ f(day, 1) &= \max(price[day] + f(day+1, 0), f(day+1, 1)) \quad (\text{holding: sell or wait}). \end{aligned}$$

Buying subtracts the price (a cost paid now); selling adds it (a reward gained now); waiting simply carries the current holding-state forward by one day unchanged. The base case is always $f(n, \cdot) = 0$ (no days left, no further profit possible).

What Changes Between the Six Variants in This Chapter

- **Buy and Sell I (Section 1.2):** only one transaction allowed – selling transitions to a terminal “done” state contributing 0 further, instead of back to $f(day+1, 0)$.
- **Buy and Sell II (Section 1.3):** the base recurrence above, unmodified – unlimited transactions.
- **Buy and Sell III/IV (Sections 1.4–1.5):** add a transaction-count dimension to cap the number of buy/sell cycles at 2 (III) or a general k (IV).
- **With Cooldown (Section 1.6):** selling jumps two days ahead instead of one, enforcing a mandatory wait before the next buy.
- **With Transaction Fee (Section 1.7):** selling subtracts a fixed fee from the reward.

Every variant is this same two-line recurrence with exactly one clause adjusted – read each section looking specifically for which single clause changed, rather than re-deriving the whole table from scratch.

1.2 Best Time to Buy and Sell Stock I

Problem Statement. Given an array *price* where *price[i]* is a stock's price on day *i*, find the maximum profit from **at most one** buy followed by one sell (the buy must happen before the sell).

Pattern Recognition

“At most one transaction” is the simplest member of Section 1.1’s holding-state family: selling is a **terminal** action here, since no second buy is ever allowed afterward.

Deriving the recurrence

Using the base template, the only change is the **sell** clause: selling on day *i* ends all activity for good, contributing exactly *price[i]* and nothing more (not $f(i+1, 0)$, which would incorrectly allow a second buy):

$$f(i, \text{holding}) = \begin{cases} \max(-\text{price}[i] + f(i+1, 1), f(i+1, 0)) & \text{not holding,} \\ \max(\text{price}[i], f(i+1, 1)) & \text{holding.} \end{cases}$$

Base case: $f(n, \cdot) = 0$.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock1Recursive(int i, int holding, vector<int>& price) {
5     int n = price.size();
6     if (i == n) return 0;
7
8     if (holding) {
9         return max(price[i], stock1Recursive(i + 1, 1, price));
10    }
11    return max(-price[i] + stock1Recursive(i + 1, 1, price),
12              stock1Recursive(i + 1, 0, price));
13 }
14
15 int main() {
16     vector<int> price = {7, 1, 5, 3, 6, 4};
17     cout << stock1Recursive(0, 0, price) << endl; // 5
18     return 0;
19 }
```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
```

```

4  int stock1Memo(int i, int holding, vector<int>& price, vector<vector<
    int>>& memo) {
5      int n = price.size();
6      if (i == n) return 0;
7      if (memo[i][holding] != -1) return memo[i][holding];
8
9      int result;
10     if (holding) {
11         result = max(price[i], stock1Memo(i + 1, 1, price, memo));
12     } else {
13         result = max(-price[i] + stock1Memo(i + 1, 1, price, memo),
14                     stock1Memo(i + 1, 0, price, memo));
15     }
16     return memo[i][holding] = result;
17 }
18
19 int main() {
20     vector<int> price = {7, 1, 5, 3, 6, 4};
21     int n = price.size();
22     vector<vector<int>> memo(n, vector<int>(2, -1));
23     cout << stock1Memo(0, 0, price, memo) << endl; // 5
24     return 0;
25 }

```

Approach 2 Complexity

Time. $O(n)$ (only 2 values of *holding*). **Space.** $O(n)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int stock1Tabulation(vector<int>& price) {
5      int n = price.size();
6      vector<vector<int>> dp(n + 1, vector<int>(2, 0));
7
8      for (int i = n - 1; i >= 0; i--) {
9          dp[i][1] = max(price[i], dp[i + 1][1]);
10         dp[i][0] = max(-price[i] + dp[i + 1][1], dp[i + 1][0]);
11     }
12     return dp[0][0];
13 }
14
15 int main() {
16     vector<int> price = {7, 1, 5, 3, 6, 4};
17     cout << stock1Tabulation(price) << endl; // 5
18     return 0;
19 }

```

Dry run of the tabulation table

Dry Run. Take $price = [7, 1, 5, 3, 6, 4]$.

Filling backward from day 5: $dp[5][1] = \max(4, 0) = 4$, $dp[5][0] = \max(-4 + 0, 0) = 0$ (buying on the last day with no chance to sell is never better than doing nothing, so this clause

$-4 + dp[6][1] = -4 + 0 = -4$ loses to 0). Continuing backward, by day 1: $dp[1][1] = \max(1, \dots)$ tracks the best sell-from-here value, and $dp[1][0] = \max(-1 + dp[2][1], \dots)$ captures buying at price 1 then later selling at the best subsequent holding value, 6, giving profit 5. The final answer $dp[0][0] = 5$ matches buying at price 1 (day 1) and selling at price 6 (day 4).

Approach 3 Complexity

Time. $O(n)$. **Space.** $O(n)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock1SpaceOptimized(vector<int>& price) {
5     int n = price.size();
6     int nextHold = 0, nextNotHold = 0;
7
8     for (int i = n - 1; i >= 0; i--) {
9         int curHold = max(price[i], nextHold);
10        int curNotHold = max(-price[i] + nextHold, nextNotHold);
11        nextHold = curHold;
12        nextNotHold = curNotHold;
13    }
14    return nextNotHold;
15 }
16
17 int main() {
18     vector<int> price = {7, 1, 5, 3, 6, 4};
19     cout << stock1SpaceOptimized(price) << endl; // 5
20     return 0;
21 }
```

Approach 4 Complexity

Time. $O(n)$. **Space.** $O(1)$ – just two scalars per day, since the *holding* dimension already has only 2 values.

The Even Simpler Greedy Alternative

Because only one transaction is allowed, this specific variant also admits an $O(n)$, $O(1)$ greedy one-pass solution: track the minimum price seen so far while scanning left to right, and at each day compute $price[i] - minSoFar$, keeping the running maximum. This works only because there is no transaction-count or cooldown complexity to track – the DP formulation above is what generalizes to every other variant in this chapter, which is why it is presented first.

Key Takeaway

The holding-state DP template, even when a simpler greedy alternative exists for this specific variant (as it does here), is worth learning first specifically because it is the version that survives every additional constraint the rest of this chapter introduces.

1.3 Best Time to Buy and Sell Stock II

Problem Statement. Given *price*, find the maximum profit from **any number** of transactions (buy then sell, buy then sell, ...), as long as you are never holding more than one share at a time (you must sell before buying again).

Pattern Recognition

“Unlimited transactions” is Section 1.1’s base recurrence, completely **unmodified**: selling returns to $f(i+1, 0)$, leaving the door open for another buy later, exactly as the general template already specifies.

The recurrence

$$f(i, \text{holding}) = \begin{cases} \max(-\text{price}[i] + f(i+1, 1), f(i+1, 0)) & \text{not holding,} \\ \max(\text{price}[i] + f(i+1, 0), f(i+1, 1)) & \text{holding.} \end{cases}$$

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock2Recursive(int i, int holding, vector<int>& price) {
5     int n = price.size();
6     if (i == n) return 0;
7
8     if (holding) {
9         return max(price[i] + stock2Recursive(i + 1, 0, price),
10                    stock2Recursive(i + 1, 1, price));
11     }
12     return max(-price[i] + stock2Recursive(i + 1, 1, price),
13                stock2Recursive(i + 1, 0, price));
14 }
15
16 int main() {
17     vector<int> price = {7, 1, 5, 3, 6, 4};
18     cout << stock2Recursive(0, 0, price) << endl; // 7
19     return 0;
20 }
```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock2Memo(int i, int holding, vector<int>& price, vector<vector<
5     int>>& memo) {
6     int n = price.size();
7     if (i == n) return 0;
8     if (memo[i][holding] != -1) return memo[i][holding];
```

```

8
9     int result;
10    if (holding) {
11        result = max(price[i] + stock2Memo(i + 1, 0, price, memo),
12                    stock2Memo(i + 1, 1, price, memo));
13    } else {
14        result = max(-price[i] + stock2Memo(i + 1, 1, price, memo),
15                    stock2Memo(i + 1, 0, price, memo));
16    }
17    return memo[i][holding] = result;
18 }
19
20 int main() {
21     vector<int> price = {7, 1, 5, 3, 6, 4};
22     int n = price.size();
23     vector<vector<int>> memo(n, vector<int>(2, -1));
24     cout << stock2Memo(0, 0, price, memo) << endl; // 7
25     return 0;
26 }

```

Approach 2 Complexity

Time. $O(n)$. **Space.** $O(n)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock2Tabulation(vector<int>& price) {
5     int n = price.size();
6     vector<vector<int>> dp(n + 1, vector<int>(2, 0));
7
8     for (int i = n - 1; i >= 0; i--) {
9         dp[i][1] = max(price[i] + dp[i + 1][0], dp[i + 1][1]);
10        dp[i][0] = max(-price[i] + dp[i + 1][1], dp[i + 1][0]);
11    }
12    return dp[0][0];
13 }
14
15 int main() {
16     vector<int> price = {7, 1, 5, 3, 6, 4};
17     cout << stock2Tabulation(price) << endl; // 7
18     return 0;
19 }

```

Dry run

Dry Run. Take $price = [7, 1, 5, 3, 6, 4]$.

The optimal strategy buys at price 1 (day 1), sells at price 5 (day 2, profit 4), buys again at price 3 (day 3), sells at price 6 (day 4, profit 3): total $4 + 3 = 7$. The tabulation table, filled backward, accumulates exactly this multi-transaction total at $dp[0][0] = 7$.

Approach 3 Complexity

Time. $O(n)$. **Space.** $O(n)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock2SpaceOptimized(vector<int>& price) {
5     int n = price.size();
6     int nextHold = 0, nextNotHold = 0;
7
8     for (int i = n - 1; i >= 0; i--) {
9         int curHold = max(price[i] + nextNotHold, nextHold);
10        int curNotHold = max(-price[i] + nextHold, nextNotHold);
11        nextHold = curHold;
12        nextNotHold = curNotHold;
13    }
14    return nextNotHold;
15 }
16
17 int main() {
18     vector<int> price = {7, 1, 5, 3, 6, 4};
19     cout << stock2SpaceOptimized(price) << endl; // 7
20     return 0;
21 }

```

Approach 4 Complexity

Time. $O(n)$. **Space.** $O(1)$.

The Greedy Alternative, and Why It Works Here Specifically

Unlimited, uncapped transactions also admit a one-line greedy solution: sum every positive day-to-day price increase, $\sum_i \max(0, \text{price}[i] - \text{price}[i-1])$. This works because, with no cap on transaction count, capturing every single upward price movement (buying right before each rise, selling right after) is always achievable and always optimal – a luxury the capped variants (III, IV) and the constrained variants (cooldown, fee) do not have, which is exactly why they need the full DP treatment.

Key Takeaway

Stock II's recurrence *is* Section 1.1's base template with nothing changed – it exists in this chapter specifically to demonstrate the unmodified template before every later section perturbs it.

1.4 Best Time to Buy and Sell Stock III

Problem Statement. Given *price*, find the maximum profit from **at most 2** transactions.

Pattern Recognition

“At most 2 transactions” requires a **new dimension** added to Section 1.1's template: a single *holding* bit can no longer distinguish “this is my first buy/sell cycle” from “this is my second,” so we add a *transactionState* counter tracking exactly which of the four actions (buy1, sell1, buy2, sell2) comes next.

Deriving the recurrence

Encode $cap \in \{1, 2, 3, 4\}$: $cap = 1$ means “next action is buy 1,” 2 means “next is sell 1,” 3 means “next is buy 2,” 4 means “next is sell 2.” Odd cap values are buy-turns; even values are sell-turns:

$$f(i, cap) = \begin{cases} \max(-price[i] + f(i+1, cap+1), f(i+1, cap)) & \text{cap odd (buy turn),} \\ \max(price[i] + f(i+1, cap+1), f(i+1, cap)) & \text{cap even (sell turn).} \end{cases}$$

Base case: $f(n, \cdot) = 0$, and also $f(i, 5) = 0$ for any i (both transactions already used up; no further action is possible, regardless of how many days remain).

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock3Recursive(int i, int cap, vector<int>& price) {
5     int n = price.size();
6     if (cap == 5 || i == n) return 0;
7
8     if (cap % 2 == 1) { // buy turn
9         return max(-price[i] + stock3Recursive(i + 1, cap + 1, price),
10                  stock3Recursive(i + 1, cap, price));
11     }
12     return max(price[i] + stock3Recursive(i + 1, cap + 1, price),
13              stock3Recursive(i + 1, cap, price));
14 }
15
16 int main() {
17     vector<int> price = {3, 3, 5, 0, 0, 3, 1, 4};
18     cout << stock3Recursive(0, 1, price) << endl; // 6
19     return 0;
20 }
```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock3Memo(int i, int cap, vector<int>& price, vector<vector<int>>& memo) {
5     int n = price.size();
6     if (cap == 5 || i == n) return 0;
7     if (memo[i][cap] != -1) return memo[i][cap];
8
9     int result;
10    if (cap % 2 == 1) {
11        result = max(-price[i] + stock3Memo(i + 1, cap + 1, price, memo),
12                    stock3Memo(i + 1, cap, price, memo));
13    } else {
```

```

14         result = max(price[i] + stock3Memo(i + 1, cap + 1, price, memo)
15             ,
16                 stock3Memo(i + 1, cap, price, memo));
17     }
18     return memo[i][cap] = result;
19 }
20 int main() {
21     vector<int> price = {3, 3, 5, 0, 0, 3, 1, 4};
22     int n = price.size();
23     vector<vector<int>> memo(n, vector<int>(6, -1));
24     cout << stock3Memo(0, 1, price, memo) << endl; // 6
25     return 0;
26 }

```

Approach 2 Complexity

Time. $O(n \cdot 4) = O(n)$. **Space.** $O(n)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int stock3Tabulation(vector<int>& price) {
5      int n = price.size();
6      vector<vector<int>> dp(n + 1, vector<int>(6, 0));
7
8      for (int i = n - 1; i >= 0; i--) {
9          for (int cap = 4; cap >= 1; cap--) {
10             if (cap % 2 == 1) {
11                 dp[i][cap] = max(-price[i] + dp[i + 1][cap + 1], dp[i +
12                     1][cap]);
13             } else {
14                 dp[i][cap] = max(price[i] + dp[i + 1][cap + 1], dp[i +
15                     1][cap]);
16             }
17         }
18     }
19     return dp[0][1];
20 }
21
22 int main() {
23     vector<int> price = {3, 3, 5, 0, 0, 3, 1, 4};
24     cout << stock3Tabulation(price) << endl; // 6
25     return 0;
26 }

```

Dry run

Dry Run. Take $price = [3, 3, 5, 0, 0, 3, 1, 4]$.

The optimal two transactions: buy at price 0 (day 3 or 4), sell at price 3 (day 5, profit 3), buy at price 1 (day 6), sell at price 4 (day 7, profit 3): total $3 + 3 = 6$, matching $dp[0][1] = 6$.

Approach 3 Complexity

Time. $O(n)$ (4 cap values per day). **Space.** $O(n)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock3SpaceOptimized(vector<int>& price) {
5     int n = price.size();
6     vector<int> next(6, 0), current(6, 0);
7
8     for (int i = n - 1; i >= 0; i--) {
9         for (int cap = 4; cap >= 1; cap--) {
10             if (cap % 2 == 1) {
11                 current[cap] = max(-price[i] + next[cap + 1], next[cap
12                     ]);
13             } else {
14                 current[cap] = max(price[i] + next[cap + 1], next[cap])
15                 ;
16             }
17         }
18         next = current;
19     }
20     return next[1];
21 }
22
23 int main() {
24     vector<int> price = {3, 3, 5, 0, 0, 3, 1, 4};
25     cout << stock3SpaceOptimized(price) << endl; // 6
26     return 0;
27 }
```

Approach 4 Complexity

Time. $O(n)$. **Space.** $O(1)$ – a fixed-size array of 6 entries per day.

Key Takeaway

Capping the number of transactions adds exactly one new state dimension (here, 4 values, encoding “which of the 4 actions comes next”), with the buy/sell branching logic itself unchanged from Section 1.1’s template – only which array slot each branch writes into and reads from has shifted. This same “add a counter dimension to cap a repeated action” idea generalizes directly to Stock IV’s arbitrary k .

1.5 Best Time to Buy and Sell Stock IV

Problem Statement. Given *price* and an integer k , find the maximum profit from **at most** k transactions.

Pattern Recognition

This generalizes Section 1.4’s fixed cap of 2 to an arbitrary parameter k . Rather than extending the 4-value parity-cap trick to $2k$ values (which works but obscures the meaning

of each state), it is clearer to track **transactions completed so far** directly as a third dimension alongside *holding*.

Deriving the recurrence

Let $f(i, used, holding)$ be the maximum profit from day i onward, having already completed $used$ transactions (a transaction counts as “completed” the moment its sell happens):

$$f(i, used, holding) = \begin{cases} \max(price[i] + f(i+1, used+1, 0), f(i+1, used, 1)) & \text{holding,} \\ \max(-price[i] + f(i+1, used, 1), f(i+1, used, 0)) & \text{not holding.} \end{cases}$$

Base cases: $f(n, \cdot, \cdot) = 0$ (no days left), and $f(i, k, \cdot) = 0$ for any i (the transaction budget is exhausted, so no further buy is permitted – selling is also moot since you cannot be holding with $used = k$ and a remaining buy budget of 0 simultaneously in any reachable state).

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock4Recursive(int i, int used, int holding, int k, vector<int>&
  price) {
5     int n = price.size();
6     if (i == n || used == k) return 0;
7
8     if (holding) {
9         return max(price[i] + stock4Recursive(i + 1, used + 1, 0, k,
10            price),
11            stock4Recursive(i + 1, used, 1, k, price));
12     }
13     return max(-price[i] + stock4Recursive(i + 1, used, 1, k, price),
14        stock4Recursive(i + 1, used, 0, k, price));
15 }
16
17 int main() {
18     vector<int> price = {3, 2, 6, 5, 0, 3};
19     int k = 2;
20     cout << stock4Recursive(0, 0, 0, k, price) << endl; // 7
21     return 0;
22 }
```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock4Memo(int i, int used, int holding, int k, vector<int>& price,
5     vector<vector<vector<int>>>& memo) {
```

```

6   int n = price.size();
7   if (i == n || used == k) return 0;
8   if (memo[i][used][holding] != -1) return memo[i][used][holding];
9
10  int result;
11  if (holding) {
12      result = max(price[i] + stock4Memo(i + 1, used + 1, 0, k, price
13          , memo),
14                  stock4Memo(i + 1, used, 1, k, price, memo));
15  } else {
16      result = max(-price[i] + stock4Memo(i + 1, used, 1, k, price,
17          memo),
18                  stock4Memo(i + 1, used, 0, k, price, memo));
19  }
20  return memo[i][used][holding] = result;
21 }
22
23 int main() {
24     vector<int> price = {3, 2, 6, 5, 0, 3};
25     int n = price.size(), k = 2;
26     vector<vector<vector<int>>> memo(n, vector<vector<int>>(k, vector<
27         int>(2, -1)));
28     cout << stock4Memo(0, 0, 0, k, price, memo) << endl; // 7
29     return 0;
30 }

```

Approach 2 Complexity

Time. $O(n \cdot k)$. **Space.** $O(n \cdot k)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int stock4Tabulation(vector<int>& price, int k) {
5      int n = price.size();
6      vector<vector<vector<int>>> dp(n + 1, vector<vector<int>>(k + 1,
7          vector<int>(2, 0)));
8
9      for (int i = n - 1; i >= 0; i--) {
10         for (int used = k - 1; used >= 0; used--) {
11             dp[i][used][1] = max(price[i] + dp[i + 1][used + 1][0], dp[
12                 i + 1][used][1]);
13             dp[i][used][0] = max(-price[i] + dp[i + 1][used][1], dp[i +
14                 1][used][0]);
15         }
16     }
17     return dp[0][0][0];
18 }
19
20 int main() {
21     vector<int> price = {3, 2, 6, 5, 0, 3};
22     cout << stock4Tabulation(price, 2) << endl; // 7
23     return 0;
24 }

```


Dry run

Dry Run. Take $price = [3, 2, 6, 5, 0, 3]$, $k = 2$.

The optimal two transactions: buy at 2 (day 1), sell at 6 (day 2, profit 4), buy at 0 (day 4), sell at 3 (day 5, profit 3): total $4 + 3 = 7$, matching $dp[0][0][0] = 7$.

Approach 3 Complexity

Time. $O(n \cdot k)$. **Space.** $O(n \cdot k)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock4SpaceOptimized(vector<int>& price, int k) {
5     int n = price.size();
6     vector<vector<int>> next(k + 1, vector<int>(2, 0)), current(k + 1,
7         vector<int>(2, 0));
8
9     for (int i = n - 1; i >= 0; i--) {
10         for (int used = k - 1; used >= 0; used--) {
11             current[used][1] = max(price[i] + next[used + 1][0], next[
12                 used][1]);
13             current[used][0] = max(-price[i] + next[used][1], next[used
14                 ][0]);
15         }
16         next = current;
17     }
18     return next[0][0];
19 }
20
21 int main() {
22     vector<int> price = {3, 2, 6, 5, 0, 3};
23     cout << stock4SpaceOptimized(price, 2) << endl; // 7
24     return 0;
25 }
```

Approach 4 Complexity

Time. $O(n \cdot k)$. **Space.** $O(k)$ for two rolling 2D slices.

Key Takeaway

Stock IV subsumes both Stock I ($k = 1$) and Stock III ($k = 2$) as special cases of the exact same (*day, used, holding*) table – a clean illustration of why generalizing a fixed-constant dimension (here, the transaction cap) into a parameter is usually worth doing directly, rather than maintaining separate hand-specialized versions for each fixed value.

1.6 Best Time to Buy and Sell Stock with Cooldown

Problem Statement. Given $price$, find the maximum profit from unlimited transactions, with one restriction: after selling, you must wait at least one full day (a **cooldown**) before buying again.

Pattern Recognition

“Cooldown after selling” modifies exactly one transition in Section 1.1’s base template: selling no longer leads to $f(i+1, 0)$ – it leads to $f(i+2, 0)$, skipping the mandatory cooldown day entirely.

Deriving the recurrence

$$f(i, \text{holding}) = \begin{cases} \max(-\text{price}[i] + f(i+1, 1), f(i+1, 0)) & \text{not holding,} \\ \max(\text{price}[i] + f(i+2, 0), f(i+1, 1)) & \text{holding.} \end{cases}$$

Only the sell clause changed: $f(i+1, 0) \rightarrow f(i+2, 0)$.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock5Recursive(int i, int holding, vector<int>& price) {
5     int n = price.size();
6     if (i >= n) return 0;
7
8     if (holding) {
9         int sell = price[i] + stock5Recursive(i + 2, 0, price);
10        int hold = stock5Recursive(i + 1, 1, price);
11        return max(sell, hold);
12    }
13    int buy = -price[i] + stock5Recursive(i + 1, 1, price);
14    int skip = stock5Recursive(i + 1, 0, price);
15    return max(buy, skip);
16 }
17
18 int main() {
19     vector<int> price = {1, 2, 3, 0, 2};
20     cout << stock5Recursive(0, 0, price) << endl; // 3
21     return 0;
22 }
```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock5Memo(int i, int holding, vector<int>& price, vector<vector<
5     int>>& memo) {
6     int n = price.size();
7     if (i >= n) return 0;
8     if (memo[i][holding] != -1) return memo[i][holding];
9
10    int result;
```

```

10     if (holding) {
11         result = max(price[i] + stock5Memo(i + 2, 0, price, memo),
12                     stock5Memo(i + 1, 1, price, memo));
13     } else {
14         result = max(-price[i] + stock5Memo(i + 1, 1, price, memo),
15                     stock5Memo(i + 1, 0, price, memo));
16     }
17     return memo[i][holding] = result;
18 }
19
20 int main() {
21     vector<int> price = {1, 2, 3, 0, 2};
22     int n = price.size();
23     vector<vector<int>> memo(n, vector<int>(2, -1));
24     cout << stock5Memo(0, 0, price, memo) << endl; // 3
25     return 0;
26 }

```

Approach 2 Complexity

Time. $O(n)$. **Space.** $O(n)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

Watch the Index Arithmetic at the Boundary

Because the sell clause now reaches two days ahead, the tabulation table needs **two** extra rows beyond n as a buffer (indices n and $n + 1$), both initialized to 0, rather than just one extra row as in every earlier section.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int stock5Tabulation(vector<int>& price) {
5      int n = price.size();
6      vector<vector<int>> dp(n + 2, vector<int>(2, 0));
7
8      for (int i = n - 1; i >= 0; i--) {
9          dp[i][1] = max(price[i] + dp[i + 2][0], dp[i + 1][1]);
10         dp[i][0] = max(-price[i] + dp[i + 1][1], dp[i + 1][0]);
11     }
12     return dp[0][0];
13 }
14
15 int main() {
16     vector<int> price = {1, 2, 3, 0, 2};
17     cout << stock5Tabulation(price) << endl; // 3
18     return 0;
19 }

```

Dry run

Dry Run. Take $price = [1, 2, 3, 0, 2]$ (days 0–4).

Filling the table backward gives $dp[0][0] = 3$, realized by two separate transactions: buy on day 0 (price 1), sell on day 1 (price 2, profit 1); cooldown on day 2; buy on day 3 (price 0),

sell on day 4 (price 2, profit 2). Total profit $1 + 2 = 3$. Note that selling on day 1 forces the next buy to wait until day 3 (day 2 is the mandatory cooldown), which the recurrence enforces automatically via its $i+2$ jump on every sell.

Approach 3 Complexity

Time. $O(n)$. **Space.** $O(n)$ table, no recursion stack.

Approach 4: Space Optimization

Now Three Rolling Rows Are Needed, Not Two

Because the sell clause reads $dp[i + 2][0]$, the space-optimized version must keep track of *two* steps of history for the not-holding value, not just one – three rolling variables (or two small arrays) instead of the usual two.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int stock5SpaceOptimized(vector<int>& price) {
5      int n = price.size();
6      int holdNext1 = 0, notHoldNext1 = 0; // i+1
7      int notHoldNext2 = 0;                // i+2
8
9      for (int i = n - 1; i >= 0; i--) {
10         int curHold = max(price[i] + notHoldNext2, holdNext1);
11         int curNotHold = max(-price[i] + holdNext1, notHoldNext1);
12
13         notHoldNext2 = notHoldNext1;
14         notHoldNext1 = curNotHold;
15         holdNext1 = curHold;
16     }
17     return notHoldNext1;
18 }
19
20 int main() {
21     vector<int> price = {1, 2, 3, 0, 2};
22     cout << stock5SpaceOptimized(price) << endl; // 3
23     return 0;
24 }
```

Approach 4 Complexity

Time. $O(n)$. **Space.** $O(1)$ – three scalars instead of two.

Key Takeaway

A cooldown (or any fixed-length mandatory delay) changes a recurrence's *lookback/lookahead distance* for one specific transition, which ripples into two places: the tabulation table needs extra boundary rows sized to the longest jump, and the space-optimized version needs to keep that many extra rows of rolling history, not just the usual one.

1.7 Best Time to Buy and Sell Stock with Transaction Fee

Problem Statement. Given *price* and a fixed *fee*, find the maximum profit from unlimited transactions, where every **completed** transaction (one buy plus one sell) costs *fee*.

Pattern Recognition

This closing problem of the chapter makes one final, equally small modification to Section 1.1's base template: the sell clause now subtracts *fee* from the proceeds, charging the cost of the completed round-trip at the moment it completes.

Deriving the recurrence

$$f(i, \text{holding}) = \begin{cases} \max(-\text{price}[i] + f(i+1, 1), f(i+1, 0)) & \text{not holding,} \\ \max(\text{price}[i] - \text{fee} + f(i+1, 0), f(i+1, 1)) & \text{holding.} \end{cases}$$

Only the sell clause changed: $\text{price}[i] \rightarrow \text{price}[i] - \text{fee}$.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock6Recursive(int i, int holding, vector<int>& price, int fee) {
5     int n = price.size();
6     if (i == n) return 0;
7
8     if (holding) {
9         return max(price[i] - fee + stock6Recursive(i + 1, 0, price,
10             fee),
11             stock6Recursive(i + 1, 1, price, fee));
12     }
13     return max(-price[i] + stock6Recursive(i + 1, 1, price, fee),
14         stock6Recursive(i + 1, 0, price, fee));
15 }
16
17 int main() {
18     vector<int> price = {1, 3, 2, 8, 4, 9};
19     int fee = 2;
20     cout << stock6Recursive(0, 0, price, fee) << endl; // 8
21     return 0;
22 }
```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock6Memo(int i, int holding, vector<int>& price, int fee,
5     vector<vector<int>>& memo) {
6     int n = price.size();
```

```

7   if (i == n) return 0;
8   if (memo[i][holding] != -1) return memo[i][holding];
9
10  int result;
11  if (holding) {
12      result = max(price[i] - fee + stock6Memo(i + 1, 0, price, fee,
13          memo),
14                  stock6Memo(i + 1, 1, price, fee, memo));
15  } else {
16      result = max(-price[i] + stock6Memo(i + 1, 1, price, fee, memo)
17          ,
18          stock6Memo(i + 1, 0, price, fee, memo));
19  }
20  return memo[i][holding] = result;
21 }
22
23 int main() {
24     vector<int> price = {1, 3, 2, 8, 4, 9};
25     int n = price.size(), fee = 2;
26     vector<vector<int>> memo(n, vector<int>(2, -1));
27     cout << stock6Memo(0, 0, price, fee, memo) << endl; // 8
28     return 0;
29 }

```

Approach 2 Complexity

Time. $O(n)$. **Space.** $O(n)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int stock6Tabulation(vector<int>& price, int fee) {
5      int n = price.size();
6      vector<vector<int>> dp(n + 1, vector<int>(2, 0));
7
8      for (int i = n - 1; i >= 0; i--) {
9          dp[i][1] = max(price[i] - fee + dp[i + 1][0], dp[i + 1][1]);
10         dp[i][0] = max(-price[i] + dp[i + 1][1], dp[i + 1][0]);
11     }
12     return dp[0][0];
13 }
14
15 int main() {
16     vector<int> price = {1, 3, 2, 8, 4, 9};
17     cout << stock6Tabulation(price, 2) << endl; // 8
18     return 0;
19 }

```

Dry run

Dry Run. Take $price = [1, 3, 2, 8, 4, 9]$, $fee = 2$.

The optimal strategy: buy at 1 (day 0), sell at 8 (day 3, proceeds $8 - 2 = 6$ after fee, profit $6 - 1 = 5$), buy at 4 (day 4), sell at 9 (day 5, proceeds $9 - 2 = 7$, profit $7 - 4 = 3$): total

$5 + 3 = 8$. Notice the fee makes the intermediate sell at price 3 (day 1) unattractive – selling and immediately rebuying would pay the fee twice for little gain, so the table correctly holds through day 1 and 2 to wait for the larger rise to 8.

Approach 3 Complexity

Time. $O(n)$. **Space.** $O(n)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int stock6SpaceOptimized(vector<int>& price, int fee) {
5     int n = price.size();
6     int nextHold = 0, nextNotHold = 0;
7
8     for (int i = n - 1; i >= 0; i--) {
9         int curHold = max(price[i] - fee + nextNotHold, nextHold);
10        int curNotHold = max(-price[i] + nextHold, nextNotHold);
11        nextHold = curHold;
12        nextNotHold = curNotHold;
13    }
14    return nextNotHold;
15 }
16
17 int main() {
18     vector<int> price = {1, 3, 2, 8, 4, 9};
19     cout << stock6SpaceOptimized(price, 2) << endl; // 8
20     return 0;
21 }

```

Approach 4 Complexity

Time. $O(n)$. **Space.** $O(1)$.

Key Takeaway

This chapter's six variants, viewed together, demonstrate exactly how far a single well-chosen state (*day, holding*, plus an optional transaction counter) can stretch: a transaction cap adds a dimension; a cooldown changes a jump distance; a fee changes a single arithmetic term. None of the six problems needed a genuinely new way of thinking about the state – each is the same holding-state machine, read closely for which single detail the problem statement perturbed.